

Project 2: Restoring Balance

Instructor: Sadab Hafiz

Due: July 05, 2026



Introduction

The Binary Search Tree (BST) is a foundational data structure that enables $O(\log(n))$ search, insertion, and deletion operations when properly balanced. However, standard BSTs can easily degenerate into inefficient, $O(n)$ linear structures when data is inserted sequentially. To overcome this limitation, self-balancing variants like the AVL tree enforce height balancing by automatically restructuring themselves during insertions and removals. In this project, you will first implement a standard BST class managing key-value pairs with modern memory management. After that, you will implement an AVL tree class that inherits from your BST class and adds height balancing. Additionally, this assignment will test your mastery of advanced C++ principles, including template inheritance, runtime polymorphism, smart pointer usage (`std::unique_ptr`), and understanding of the Big Five copy/move semantics. You will also be introduced to a post function hook pattern which you will use for height updates and balancing.

Beyond data structure design, this project emphasizes runtime performance analysis. You are provided with a `main.cpp` file which inserts sequential keys into a BST and AVL tree showing time and height differences between the two. After you implement your trees correctly, it will be evident why height balancing is very important in context of search tree usage. This will be a great learning experience!

Setting up GitHub

This step is necessary to get started with the project. You will use a personal, private Git repository to manage your codebase and track your progress. To get started, visit the [GitHub website](#) and sign up for a free account if you do not have one already. After creating your account, I highly encourage you to watch [this video by Brian Yu](#) which covers the basics of Git and GitHub.

Repository Setup Instructions:

1. Create a **new repository** on your GitHub account. Name it CSCI335-AVLTree.
2. **IMPORTANT:** You must set your repository visibility to **Private**. Making your repository public will expose your solution to other students, which is a **violation of the academic integrity** policy.
3. Download the project starter code from **Brightspace**, initialize a git repository locally (`git init`), link it to your GitHub private remote, and push your initial commit.

When it is time to submit, log into Gradescope, select this assignment, and choose the option to submit via GitHub. Connect your GitHub account and select your private repository. **Do not create forks of any public tracking repositories** to ensure your development history remains private. You can find submission instructions [here](#). Make sure to refer back to these instructions when it's time to submit.

Documentation and Design

Documentation and design together are worth 15% of each project grade. The requirements are:

Documentation Requirements (10%)

- All files must begin with a top-level block comment containing at least your name, the date, and a brief description summarizing the purpose of the code implemented in that file.
- All functions in the `.hpp` files must include a structured Doxygen-style block comment:

```
/**
 * @brief Describes what the function does in general terms.
 *
 * @pre Describes any preconditions that must be true before calling.
 * @param Description of an individual parameter (repeat for each parameter).
 * @return Description of the returned value and type.
 * @post Describes postconditions or side-effects after execution.
 * @throws Describes any exceptions the function may throw.
 */
```

For the required project functions, these blocks are provided in your starter instructions. However, you are strictly required to write them for any additional helper functions you choose to implement.

- While header documentation describes interface usage, you must use inline comments (`//`) inside your `.cpp` files to clarify complex implementation details (such as loops, intricate logic, or conditionals):

```
// Traverse down the tree structure to find the target key
for (DataType x : some_arr_) {
    // If a matching key is found, update the corresponding value
    if (...condition...) {
        ...implementation code...
```

```

    }
    else {
        ...implementation code...
    }
}

```

If a block of code is not immediately obvious to an outside reader, leave a brief inline comment. Do not comment every single line. Self-explanatory code should remain uncommented.

Design Requirements (5%)

- You should follow the naming convention below:

```

string my_variable;           // Snake_case for local variables
class MyClass {};            // PascalCase for class names
MyClass class_instance;      // Snake_case for instances
string my_data_member_;      // Trailing underscore for private members
void myFunction();           // camelCase for member functions
const int MY_CONSTANT = 10;  // ALL_CAPS for constants

```

- Maintain architectural consistency throughout your codebase. This applies to your choice of include guards (`#pragma once` vs. `#ifndef`), member ordering inside classes, and general formatting choices.
- Adhere strictly to Object-Oriented Programming (OOP) principles. Public data members and global variables are prohibited unless explicitly authorized by the project specification.
- Use the `const` keyword and pass-by-reference (`&`) design patterns appropriately. Functions that do not modify private class data members must be marked `const`.
- Do not include `using namespace std;` in any header (`.hpp`) files or any files in case of class templates.
- **Resource and Memory Management:** Ensure all dynamically allocated blocks are explicitly freed or cleanly managed using appropriate smart pointers. If a class overrides a custom destructor, ensure the remaining Rule of Five operators are correctly handled.

Helpful Tips

If you are developing in VS Code, I highly recommend utilizing extensions to automate your documentation generation. The starter frameworks provided use templates compatible with [Doxygen](#), a tool standard for auto-generating C++ documentation. Consider installing the following extensions:

- [Doxygen Documentation Generator](#) : Automatically generates structured comment block skeletons.
- [C/C++ for VSCode](#) : Official extension providing IntelliSense autocomplete, syntax formatting, etc.

If you lack a working local C++ environment, reach out immediately. The provided starter header templates serve as a direct visual baseline for your documentation requirements.

Starter Code Tips

In the provided files, make sure you address each `TODO` comment and delete the comments after addressing them. If you plan to modify a function that is explicitly stated not to be modified, make a copy of the original. You can also look at the original starter files from GitHub commit history.

Task A: BST Class



Your first task is to implement the templated `BST` class. The header file is available in the starter code. Carefully go through each of the function comments to figure out how to implement them. Some functions can be used to implement others, and I encourage you to read the entire header before writing any code. To assist you, the `BST.cpp` starter file includes fully implemented `insert` and `remove` entrance functions, alongside structural scaffolding for the insert and remove helper functions. **Only address the `TODO` comments in the provided helper functions and don't modify the entrance functions.** Study the provided code closely as they demonstrate how the recursive helper functions interact and show how success paths flow through the post-operation function for height updates.

Tips for implementation:

- **handlePost:** This virtual function acts as a post-operation intercept on the way back up the recursive call stack. In this task, you will use it to recalculate and update the current node's height field based on its children. Task B will rely on overriding this exact hook to execute AVL balancing algorithms.
- **Deep Copy:** You are responsible for implementing the copy constructor and copy assignment operator from scratch. To do this recursively, you will need to declare and implement your own recursive copy helper function that allocates completely new nodes in memory.
- **Iterative Logic:** The retrieval and searching operations (`contains`, `operator[]`, `getInOrderKeys`, and `getInOrderValues`) should be implemented **iteratively** rather than recursively.

A `main.cpp` file is provided to benchmark insertion times and final tree heights. You can comment out the AVL tree bits to test insertion. While it acts as a great validation tool, it is basic and does not check all cases and functions. You are expected to expand this file or create your own tests to verify your implementation. **You are NOT allowed to use any external data structure libraries (`std::map`, `std::set`, etc.)**

Task B: AVL Class



Your second task is to implement the templated `AVL` class. This class inherits from `BST` and implements height balancing by overriding the post-operation function hook `handlePost`. To assist you, the starter `AVL.cpp` file includes a fully functional `rotateRight` function and the left-heavy logic inside `rebalance`. **Do not modify the provided rotation logic.** Use it as a reference for how to cleanly shift node pointers and transfer `std::unique_ptr` ownership.

Tips for Implementation:

- **handlePost:** When overriding this function, your first line should invoke the base class function to update the current node's height. Because `AVL` inherits from a class template, remember to explicitly scope your call using `BST<K, V>::` to avoid an infinite recursive loop.
- **Right-Heavy Balancing:** Complete the remaining branches of the `rebalance` function to cover the right-heavy cases (right-right and right-left imbalances).
- **Remaining Functions:** You are responsible for implementing the rest of the functions by looking at expected functionality as stated in `AVL.hpp`.

Once you have finished your balancing mechanics, uncomment the `AVL` tree blocks in `main.cpp` and run the benchmark. If your insert function works correctly, you will see a tree of height 13. You are responsible to test the rebalancing of your tree after insert and remove operations.

Compiling and Testing

Since AVL requires BST to be implemented correctly, I recommend implementing and testing the BST class first before moving on to AVL. Assuming you implement everything, you can compile everything with `g++ -std=c++14 main.cpp`. Note that we do not compile `BST.cpp` or `AVL.cpp` as they are class templates. Including them in `main.cpp` will cause them to be compiled automatically.

Make sure to test all functions and edge cases by writing tests to ensure they work as specified. Testing is a very important skill that you need to learn by trial and error. If you get an error, look at the error message and find out which line it is coming from. I recommend implementing each function and testing it before implementing the next one.

Submissions

You will submit your code to Gradescope directly from your private GitHub repository. Refer back to the **Setting up GitHub** section of this document to review the instructions and videos if you haven't done so already. The autograder will grade: `BST.hpp`, `BST.cpp`, `AVL.hpp`, and `AVL.cpp`. Although Gradescope allows multiple submissions, it is not a place to test your code every time you make changes. You have to test it locally before uploading to Gradescope. To prevent the abuse of the autograder and to encourage testing, **the autograder feedback will be limited**. Your code needs to compile on Gradescope in order to get any feedback from the autograder. Furthermore, the autograder will be released near the due date which incentivizes testing your own code instead of relying on the autograder. Once you pass the test cases on Gradescope, make sure to verify the documentation and design requirements for full credit.

Plagiarism and Citing

Your code in all the `.cpp` files should not 100% match with another student. If that happens, I will assume it is a violation of academic integrity. It can happen with the use of generative AI, it can also happen if you both copy code from the same source. If you use libraries or functions that are not mentioned in the slides, or I didn't teach in class, you have to cite them with an inline comment. You should also be able to explain such lines of code if I ask you to explain what you wrote. In general, if you are not sure if something is appropriate for the project, you can reach out to confirm. Any proven academic dishonesty will result in a significant penalty to your assignment grade depending on the violation.

Additional Tips

Here are more tips to help you. If more clarification is needed, they will be announced on **Brightspace**.

- **Accessing BST Functions:** Because the AVL class inherits publicly from `BST<K, V>`, all public and protected base class functions are inherited and fully accessible inside your AVL implementation.
- **Using `.get()`:** Several helper functions, such as `updateHeight` and `getBalanceFactor`, accept a raw pointer (`Node<K, V>*`) rather than a smart pointer reference. You can extract the underlying raw pointer managed by a `std::unique_ptr` by calling its `.get()` member function (for example, `updateHeight(current.get());`). This allows you to inspect or update a node's fields without altering or transferring its strict memory ownership.